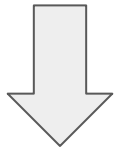
# Overflowing With Fear

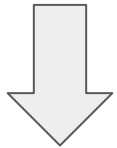
Detecting and Mitigating Implicit Panics in Rust

```
let c = a + b;
```



```
let Some(c) = a.checked_add(b) else {
        panic!();
};
```

```
let a = slice[idx];
```



```
let Some(a) = slice.get(idx) else {
        panic!();
}
```

Implicit panics inserted by the compiler without warning.

Problem: no (easy) proof that a given function does not have a mine ready to explode!

Not something we can transparently fix:

- Overflowing or out-of-bounds access is also unacceptable.
- These behaviors are here for safety reason…

… it is just that their consequences in the kernel are more dramatic.

Each panic site can be replaced by using a fallible operation:

- Potentially overflowing operation with `checked_add`,
- Potentially out-of-bounds access with `get`,
- Etc.

Problem is knowing when they can happen: opt-in clippy warning? Compiler support?

# User-space example: `no_panic`

```
#[no_panic]
fn demo(s: &str) -> &str {
    &s[1..]
}
```

Triggers a linker error if `demo` can ever unwind

- Similar mechanism to `build_assert`, relying on compiler optimizations.